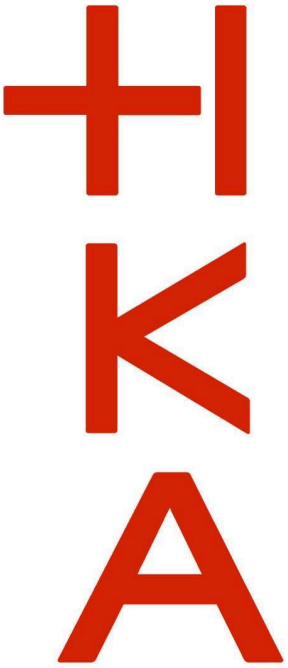
ANGEBOT

Micro-App-Plattform



KI-Apps im Intranet der Hochschule Karlsruhe

Dieses Angebot nennt Leistung, Termine und Preis für das Projekt Micro-App-Plattform (Vibe-Coding Intranet-Plattform) der Hochschule Karlsruhe.

Angebotsnummer	AWP-WS26-01
Datum	12.10.2026
Gültig bis	23.10.2026
Auftraggeber	Hochschule Karlsruhe (HKA), Prof. Dr. Rainer Neumann und Dr. Günther Schreiner
Auftragnehmer	Hochschule Karlsruhe (HKA), Moltkestraße 30, 76133 Karlsruhe (Studienteam Anwendungsprojekt WS 2026/27)
Vertragsart	Werkvertrag (§ 631 ff. BGB) mit agilem Phasenfestpreis
Grundlage	Konzeptpräsentation „Micro-App-Plattform“ (AWP 2026/27) und die Gespräche mit dem Auftraggeber

Inhalt

1 Executive Summary.....3

2 Wer wir sind und warum wir.....3

3 Ausgangslage und Auftragsverständnis.....4

4 Zielbild und Nutzen.....5

5 Lösungskonzept.....5

6 Leistungsumfang und Liefergegenstände.....10

7 Vorgehen und Projektorganisation.....12

8 Voraussetzungen, Mitwirkung und Annahmen.....13

9 Qualität, Erfolgs- und Abnahmekriterien.....14

10 Termin- und Meilensteinplan.....16

11 Vergütung.....17

12 Änderungsverfahren.....18

13 Übergabe und Weiterbetrieb.....19

14 Angebots- und Vertragsbedingungen.....19

15 Anlagen.....20

Annahmeerklärung.....21

1 Executive Summary

Die HKA baut mit KI-Assistenten Apps in Stunden statt in Monaten. Ohne gemeinsame Regeln entstehen daraus Einzellösungen: Jede App sieht anders aus, hat ein eigenes Login und läuft auf einem eigenen Rechner.

Wir liefern bis zum **29.01.2027** eine Plattform, mit der Fachexpertinnen und Fachexperten ohne Entwicklungserfahrung gemeinsam mit einem KI-Assistenten Micro-Apps bauen. Eine Micro-App ist eine kleine Webanwendung für genau einen fachlichen Zweck; die genaue Definition steht in Abschnitt 6. Die Apps sehen gleich aus, bestehen automatische Prüfungen und erscheinen nach der Freigabe im Intranet-Portal. Anmeldung, Prüfungen und Deployment laufen automatisch über die Plattform.

Die Plattform besteht aus drei Teilen:

- **Entwicklung:** Zwei Starter-Projekte („Golden Master“) für Python (FastAPI) und JavaScript (Express). Dazu kommen Regeln für die KI, damit jede neue App gleich aufgebaut ist und gleich aussieht.
- **Pipeline:** Jeder Pull Request durchläuft automatische Prüfungen. Eine zweite Person gibt ihn frei. Danach rollt die Pipeline die App aus.
- **Betrieb:** Ein Portalrahmen übernimmt Login, Rollen und Sessions. Er trägt neue Apps in das Menü ein.

Mitarbeitende der HKA-IT mit Grundkenntnissen in Docker und Gitlab können die Plattform anhand der Betriebsanleitung betreiben. Der erste Durchstich durch alle drei Teile steht am **19.11.2026**. Der Festpreis beträgt **36.000 € zuzüglich Umsatzsteuer**, abgerechnet in vier Phasen.

2 Wer wir sind und warum wir

Wir sind ein Team aus fünf Studierenden der Hochschule Karlsruhe und bearbeiten das Projekt im Anwendungsprojekt (AWP) im Wintersemester 2026/27. Wir sind keine Firma, sondern ein Projektteam. Wir haben weniger Projekterfahrung als ein etablierter Dienstleister und sagen das offen. Dafür arbeiten wir täglich mit KI-Assistenten und kennen die Probleme, die diese Plattform lösen soll, aus eigener Erfahrung.

2.1 Das Team

Name	Rolle im Projekt
Alexandru Tatar	Fullstack Entwickler, DevOps, Testen
Antonio Codrean	Backend Entwickler, Testen
David Coric	Dokumentation, Backend Entwickler, Testen
Xuan Thanh Tran	Testen, Backend Entwickler
Yaraslau Krautsou	Backend Entwickler, DevOps, Testen

2.2 Wie wir die Qualität unserer Arbeit sichern

Wir wenden auf unsere eigene Arbeit dieselben Regeln an wie die Plattform auf die Apps: automatische Prüfungen, Freigabe durch eine zweite Person, automatisierte Tests, vorab festgelegte Abnahmekriterien und ein Sprint-Review alle zwei Wochen. Die Maßnahmen stehen in Abschnitt 9.1, die Abnahmekriterien je Phase in Abschnitt 9.3.

3 Ausgangslage und Auftragsverständnis

3.1 Situation

Fachexpertinnen und Fachexperten der HKA haben in der Regel keine Entwicklungserfahrung. Mit KI-Assistenten wie Cursor, GitHub Copilot oder Claude Code können sie Apps trotzdem selbst bauen: Eine Fachexpertin schreibt der KI, was sie braucht („Bau mir eine Webanwendung für meine Hochschule“), und die App steht nach wenigen Tagen.

3.2 Probleme heute

Weil jede Fachexpertin ihre App allein baut, entstehen Einzellösungen. Die HKA hat dadurch schon heute vier Probleme:

Problem	Was heute passiert	Folge für die HKA
Verschiedene Oberflächen	Jede App sieht anders aus, nutzt andere Bibliotheken, hat zu wenig Kontrast und ist nicht barrierefrei.	Nutzende finden sich schlecht zurecht. Barrierefreiheit ist nicht gesichert.
Sicherheitsrisiken	KI-Code kann Secrets im Klartext enthalten oder Fehler bei der Anmeldung machen. Jede App baut ein eigenes Login.	Zugangsdaten und Daten können abfließen. Die Anmeldung ist uneinheitlich.
Einzellösungen	Apps laufen lokal oder auf verstreuten VMs. Niemand weiß, unter welcher URL sie erreichbar sind. Mail, Intranet und CMS sind nicht angebunden.	Apps sind schwer zu finden und nicht in die Systeme der HKA eingebunden.
Keine Konventionen	Jede App startet mit anderen Befehlen (`npm start`, `npm run web`, `python main.py`). Dokumentation fehlt oft.	Betrieb und Übernahme durch andere Personen sind über längere Zeit aufwendig.

3.3 Was die HKA erreichen will

Die HKA will die Qualität verbessern. Die KI baut die App weiter schnell. Hosting, Betrieb und Sicherheit sollen aber nicht bei der Fachkraft bleiben. Sie kennt sich damit nicht aus und sollte sich damit nicht befassen müssen.

Die Zielgruppe der Plattform sind Fachexpertinnen und Fachexperten ohne Entwicklungserfahrung. Sie beschreiben der KI, was sie brauchen, und erhalten eine App, die Regeln, Prüfungen und Betrieb schon enthält.

4 Zielbild und Nutzen

Nach dem Projekt startet eine Fachexpertin eine neue Micro-App aus einem Starter-Projekt, baut sie mit der KI und bringt sie ins Hochschul-Repository.

Die folgenden Ziele und Erfolgskriterien zeigen, woran die HKA den Nutzen erkennt. Erfolgskriterien und Abnahmekriterien sind getrennt: Abnahmekriterien zeigen, dass wir geliefert haben (siehe Abschnitte 6 und 9). Erfolgskriterien zeigen, dass die Plattform wirkt.

Die HKA hat die Probleme aus Abschnitt 3 heute schon. Die Plattform löst sie. Den Nutzen erkennt die HKA an diesen fünf Zielen:

- **Schnell starten:** Ein neues Projekt aus einem Starter läuft nach dem Klonen mit denselben fünf Befehlen (`dev`, `start`, `test`, `lint`, `docker`).
- **Gleiches Aussehen und Barrierefreiheit:** Apps aus beiden Startern folgen denselben Regeln (Design, Kontrast, einfache Sprache).
- **Sicherheit ohne Expertenwissen:** Der Secret-Scan blockiert jeden Pull Request mit erkanntem Secret. Jede Änderung gibt eine zweite Person frei.
- **Alles an einem Ort:** Nach dem Deployment erscheint die App im Portal-Menü. Login und Rollen kommen vom Portal.
- **Betrieb über längere Zeit:** Start, Stopp und Neustart laufen mit Docker Compose und sind dokumentiert.

5 Lösungskonzept

Eine App durchläuft drei Teile. Sie entsteht lokal aus einem Starter (goldenen Repository). Eine Pipeline prüft sie und rollt sie aus. Danach läuft sie im Portalrahmen, der Login und Rollen für sie übernimmt. Die Fachexpertin arbeitet nur im ersten Teil. Die Architektur zeigen die Bilder in Abschnitt 5.5.

5.1 Entwicklung

Jeder Starter enthält eine feste Projektstruktur, eine `manifest.json` und eine Konfiguration für VS Code (DevContainer).

Die Datei `AGENTS.md` steuert die KI und verweist auf den passenden Skill. Der Ordner `.agents/skills` enthält sieben Skills: design, barrierefreiheit, ui-regeln, schnittstellen, portal, pipeline-fehler und secrets. Die KI lädt `AGENTS.md` immer und einen Skill nur bei Bedarf. Sie hält Entscheidungen laufend in der README fest. So lässt sich eine App später nachvollziehen oder neu bauen.

5.2 Betrieb

Der Portalrahmen meldet Nutzer über Shibboleth (OIDC) an. Das Security-Gateway prüft jede Anfrage. Die Session liegt in Redis, der Browser erhält nur ein Cookie. Das Gateway gibt die Identität als Header an die App weiter. Die App sieht keine Tokens und keine Passwörter.

Neue Apps melden sich über ihr Manifest an (Name, Kategorie, Menüpunkt, Rollen) und erscheinen im Menübaum, zum Beispiel unter Studium, Verwaltung oder Kommunikation. Jede App läuft als eine Instanz in einem eigenen Container.

Der Betrieb umfasst den ganzen Lebenslauf einer App. Die Tabelle zeigt, was die Plattform dafür bereitstellt.

Lebenslauf	Was die Plattform bereitstellt
Starten	Nach dem Merge in `main` startet die Pipeline den Container der App auf einer HKA-VM (Docker Compose). Erst wenn der Health Check besteht, trägt sie die App im Portal ein.
Laufen	Jede App läuft als genau eine Instanz. Fällt der Container aus, startet Docker Compose ihn neu. Die Logs liest die HKA-IT mit Docker-Befehlen.
Aktualisieren	Ein neuer Merge in `main` ersetzt die laufende Version. Besteht die neue Version den Health Check nicht, meldet die Pipeline den Fehler. Die HKA-IT setzt die App dann mit einem Befehl auf die vorherige Version zurück.
Beenden	Eine App wird beendet, indem ihr Container gestoppt und ihr Manifest aus der Registry entfernt wird. Der Menüpunkt verschwindet automatisch.
Speicher	Die Daten einer App liegen in einem eigenen Docker-Volume auf der HKA-VM (Abschnitt 8.2). Die Betriebsanleitung beschreibt, wie man ein Volume sichert, wiederherstellt und beim Beenden einer App löscht. Ein automatisches Backup gehört nicht zum Leistungsumfang.
Plattform selbst	Portalrahmen, Security-Gateway und Redis laufen ebenfalls als Container mit Docker Compose. Start, Stopp und Neustart stehen in der Betriebsanleitung.

5.3 Qualitätskreislauf

Wir sichern im Projekt die Qualität der KI-gebauten Apps in einem Kreislauf aus fünf Schritten. Das Projektteam führt den Kreislauf aus. Übergeben werden die Ergebnisse: Skills, Prüfprotokolle und Prüfskript. Mit dem Prüfskript kann die HKA den Test später selbst wiederholen.

1. **Entwickeln:** Die KI baut die App aus dem Starter.
2. **Prüfen:** Die Pipeline führt alle Prüfungen aus.
3. **Simulieren:** Das Projektteam spielt Fehler durch, zum Beispiel eine rote Pipeline und ein Secret im Code. Die KI soll den Fehler erklären und die nächsten Schritte zur Lösung nennen. Ob sie das tut, hält das Projektteam im Prüfprotokoll fest.
4. **Qualität prüfen:** Die KI baut eine feste Testaufgabe, zum Beispiel „Baue eine Noten-Übersicht“. Eine Person prüft von Hand, ob die KI die sieben Regeln einhält (Abschnitt 6, Golden Master), und schreibt ein Protokoll. Das Protokoll nennt das Datum, die Version der Skills und die Testaufgabe, damit sich der Lauf wiederholen lässt. Für jede der sieben Regeln hält die Person fest, ob sie erfüllt oder verletzt ist. Zu jeder Abweichung schreibt sie, was passiert ist, in welcher Datei und Zeile es steht und welcher Skill die Regel abdecken soll. Außerdem ordnet sie jede Abweichung ein: Hat die KI die Regel nicht befolgt? Steht die Regel im Skill nicht klar genug? Fehlt sie im Skill?
5. **Qualität sichern:** Das Projektteam ergänzt die Skills gemäß Protokoll und lassen den Test erneut laufen. Aus den Protokollen entsteht ein Skript, das jede Regel bei jedem Lauf automatisch prüft. Ein

Lauf ohne Abweichungen wird als Goldener Standard gespeichert. Jede neue Skill-Version wird mit ihm verglichen.

Gegen Secrets im Code gibt es drei Sicherungen: einen Commit-Hook, die Push Protection von Gitlab und den Secret-Scan in der Pipeline.

5.4 Technologien und Standards

Wir legen diese Werkzeuge und Standards fest:

Entwicklungsumgebung: Visual Studio Code mit Erweiterungen, EditorConfig und DevContainer

Starter (Backend): Python mit FastAPI, JavaScript mit Express

Frontend: HTML und CSS nach den Gestaltungsregeln

KI-Regeln: `AGENTS.md` und Skills je Repository, nutzbar mit Cursor, GitHub Copilot oder Claude Code

Linter: Ruff (Python), ESLint (JavaScript)

Automatisierte Tests: pytest (Python), Jest (JavaScript), Ausführung in der Pipeline

Code und Pipeline: Gitlab mit CI/CD Pipelines, Secret-Scan mit Gitleaks, Manifest-Check

Container und Betrieb: Docker und Docker Compose mit Health Check auf HKA-VMs (Proxmox)

Login und Session: Shibboleth über OIDC, Session-Speicher Redis

5.5 Architektur (C4 Level 1 und 2)

Die Bilder zeigen die Architektur nach dem C4-Modell. Level 1 zeigt die Plattform in ihrer Umgebung, Level 2 die Bausteine der Plattform.

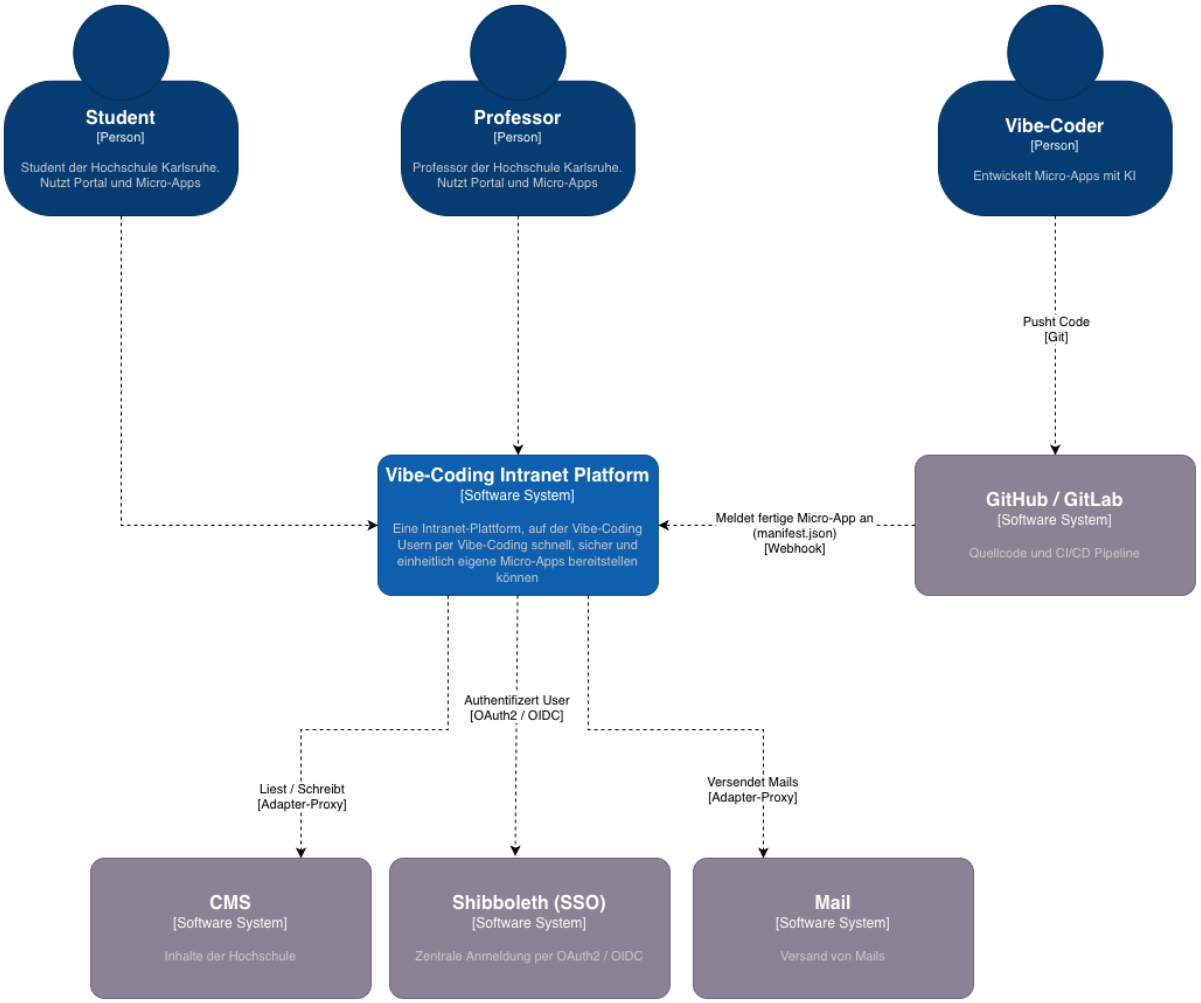


Abbildung 2: C4 Level 1 – Systemkontext der Micro-App-Plattform

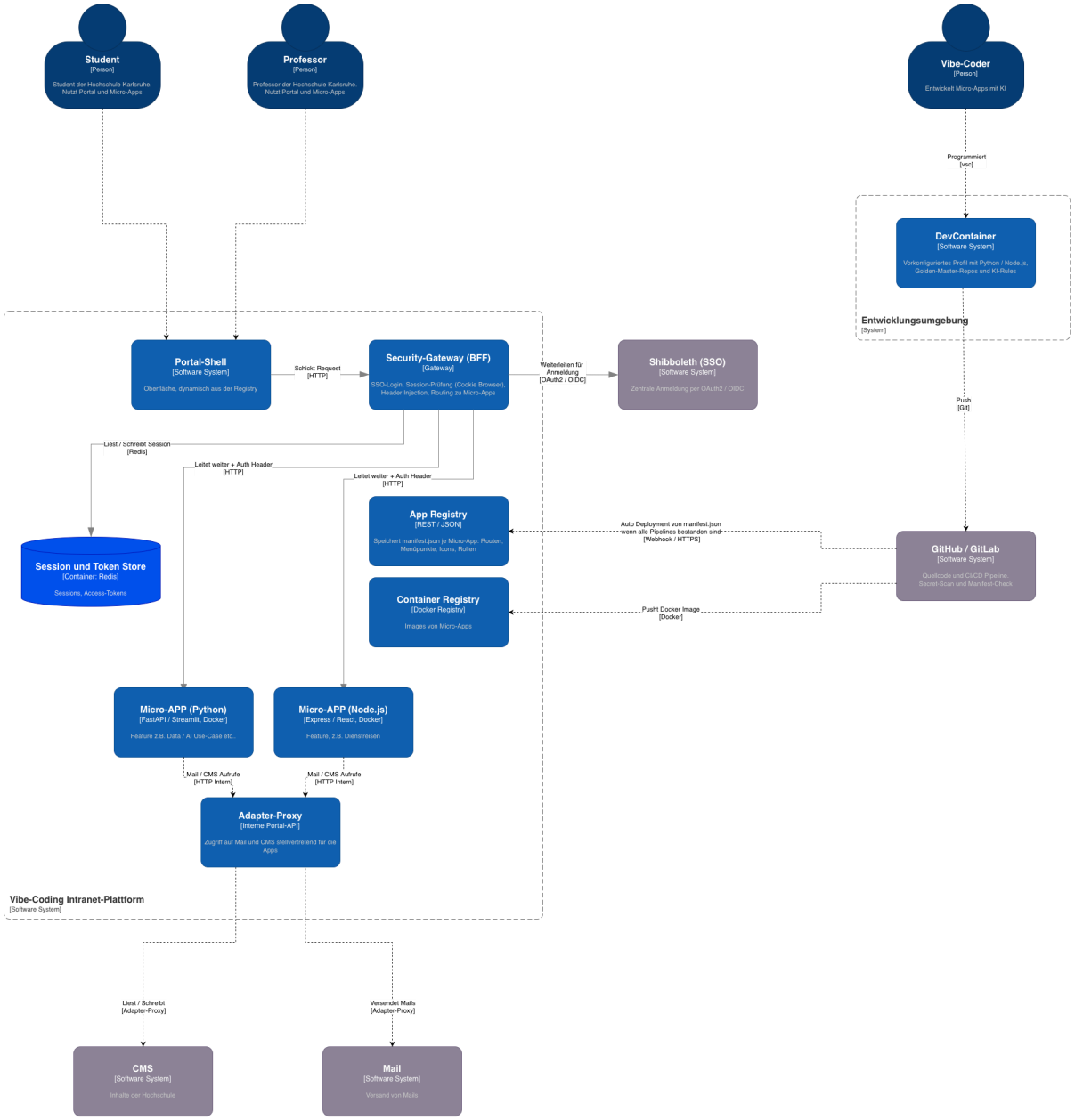


Abbildung 3: C4 Level 2 – Bausteine der Micro-App-Plattform

5.6 Geplante Referenz-Apps (Entwurf)

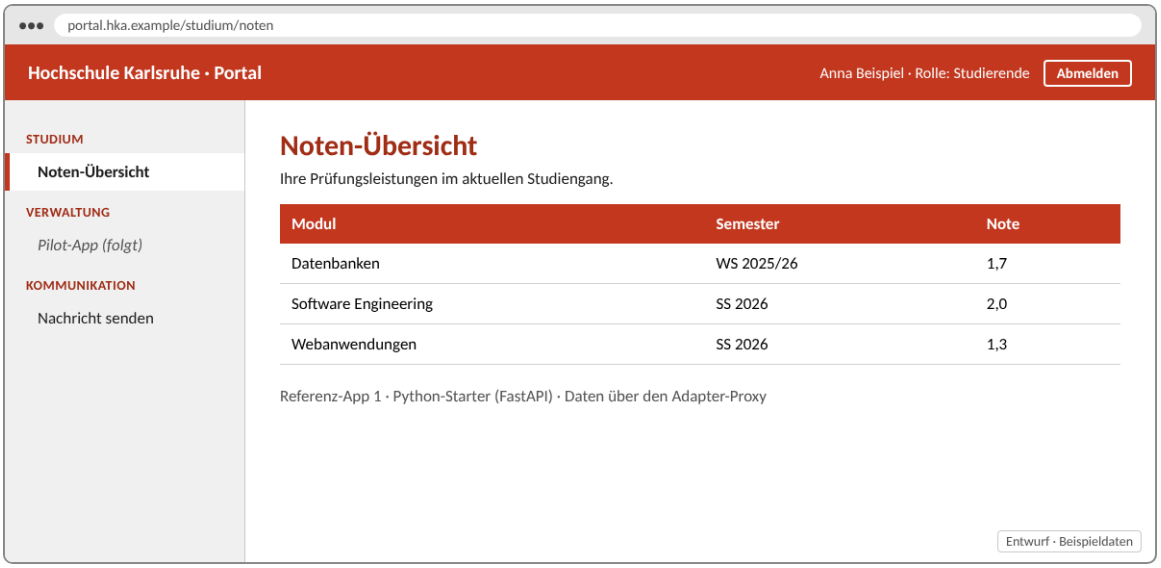


Abbildung 4: Entwurf Referenz-App 1 (Python-Starter) – Noten-Übersicht im Portalrahmen

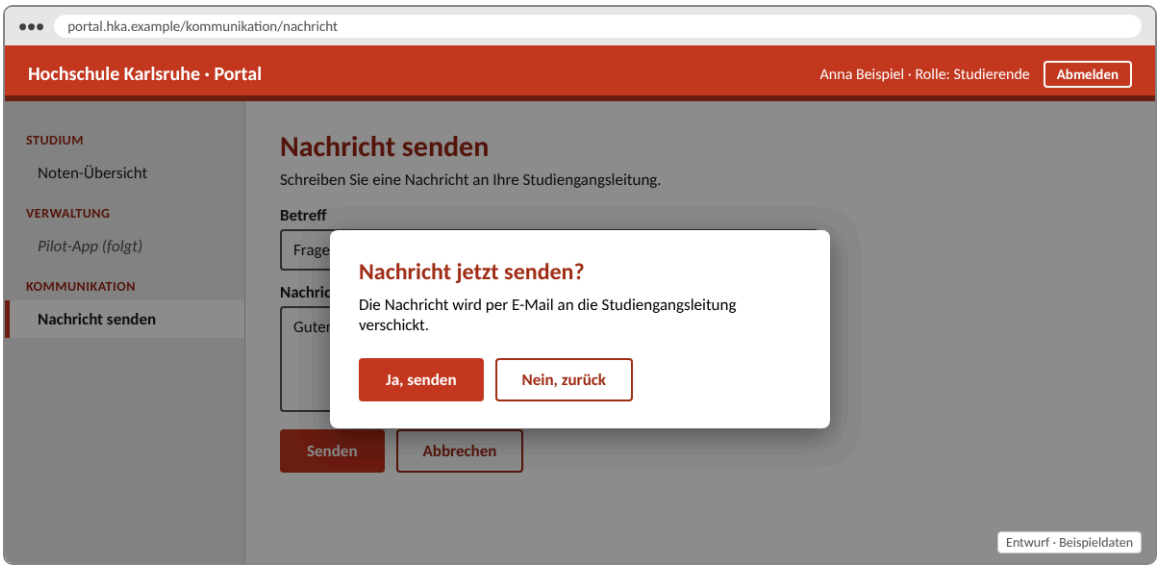


Abbildung 5: Entwurf Referenz-App 2 (JavaScript-Starter) – Nachricht senden mit Dialog mit zwei Buttons

6 Leistungsumfang und Liefergegenstände

Wir liefern vier Arbeitspakete, zwei Referenz-Apps, einen Pilot und die Dokumentation. Die Pakete laufen parallel, damit früh ein Durchstich durch alle Teile entsteht. Zu jedem Liefergegenstand nennen wir den Nachweis für die Abnahme. Die Abnahmenachweise gelten zusammen mit den Abnahmekriterien je Phase (Abschnitt 9.3).

Portal

Ergebnis: Portalrahmen mit Menübaum nach Kategorie, Registry für App-Manifeste (`manifest.json`) und Aufnahme neuer Apps in das Menü.

Abnahmenachweis: Eine registrierte App erscheint im Portal. Der Menüpunkt erscheint nur für Personen mit einer Rolle aus dem Manifest. Wird das Manifest entfernt, verschwindet der Menüpunkt.

Golden Master

Ergebnis: Starter für Python (FastAPI) und JavaScript (Express), `AGENTS.md` und sieben Skills, Gestaltungsregeln auf Basis des Hochschul-CSS (Menü links, Dialog mit zwei Buttons, Kontrast, einfache Sprache), Schnittstellenbeschreibungen für Mail, Noten und E-Akte, eine Anleitung für Fachexpertinnen sowie KI-Tests mit Prüfprotokoll und Prüfskript.

Abnahmenachweis: Zwei Apps aus verschiedenen Startern folgen denselben Regeln. Die Linter laufen ohne Fehler. Das Prüfskript meldet für die Testaufgabe in beiden Startern 7 von 7 Regeln bestanden.

Pipeline

Ergebnis: Gitlab CI/CD Pipelines mit Secret-Scan, Manifest-Check, Lint, Tests und Docker-Build, Pflicht-Freigabe durch eine zweite Person, Deployment mit Docker Compose auf einer HKA-VM, Eintrag im Portal, Health Check und Neustart des Containers bei Ausfall.

Abnahmenachweis: Ein Merge rollt die App aus, und sie ist im Portal erreichbar. Eine rote Prüfung sperrt den Merge. Die App lässt sich laut Betriebsanleitung stoppen, starten und auf die vorherige Version zurücksetzen.

Sicherheit

Ergebnis: Security-Gateway, Login über Shibboleth (OIDC), Rollen, Session in Redis, Weitergabe der Identität als Header, Kommunikation mit Schnittstellen per Adapter-Proxy, drei Sicherungen gegen Secrets im Code.

Abnahmenachweis: Login mit Hochschul-Account. Eine App sieht nur die Rollen der angemeldeten Person. Ein Secret im Code blockiert den Merge.

Zwei Referenz-Apps

Ergebnis: Je eine Micro-App pro Starter. Beide laufen im Portal, folgen den sieben Regeln und können Folgendes (Entwürfe: Abschnitt 5.6):

Referenz-App	Was die App können soll
Noten-Übersicht Python-Starter	Nach dem Login zeigt die App Name und Rollen der angemeldeten Person. Sie zeigt eine Tabelle mit Modul, Semester und Note. Die Daten kommen als Testdaten über den Adapter-Proxy aus der Noten-Schnittstelle. Die App kann nur lesen. Der Logout beendet die Session.
Nachricht senden JavaScript-Starter	Nach dem Login zeigt die App Name und Rollen der angemeldeten Person. Ein Formular mit Betreff und Text. Ein Dialog mit zwei Buttons fragt vor dem Senden nach. Die Nachricht geht über den Adapter-Proxy und die Mail-Schnittstelle an eine Test-Adresse. Der Logout beendet die Session.

Abnahmenachweis: Beide Apps laufen im Portal und können das, was in der Tabelle steht. Live-Demo in der Abschlusspräsentation.

Pilot

Ergebnis: Mindestens zwei Fachexpertinnen und Fachexperten der HKA bauen mit einem Starter Apps und bringen sie ins Portal. Wir schulen sie in einem Workshop, begleiten sie im Pilotzeitraum KW 50 bis KW 3 und werten die Ergebnisse in einem Kurzbericht aus.

Abnahmenachweis: Mindestens eine App aus dem Pilot läuft im Portal, und der Kurzbericht liegt vor. Benennt die HKA keine Pilotpersonen (Abschnitt 8.1), bauen wir selbst eine Pilot-App. Die Abnahme hängt nicht vom Erfolg der Pilotpersonen ab.

Dokumentation

Ergebnis: README je Repository, Architektur (C4 Level 1 bis 2), Betriebsanleitung (Start, Stopp, Neustart, Fehlerfall, Aktualisieren und Zurücksetzen einer App, Beenden einer App, Sichern und Wiederherstellen eines Daten-Volumes), Anleitung für neue Apps und eine Liste der verwendeten Open-Source-Komponenten.

Abnahmenachweis: Übergabe als Markdown im Repository.

Abgrenzung

Nicht im Leistungsumfang enthalten sind:

- Produktivbetrieb, Wartung und Support nach der Abnahme
- Migration oder Umbau bestehender Apps der HKA
- ein Sicherheitsaudit oder Penetrationstest der Plattform
- Hardware, Lizenzen und Kosten für KI-Dienste

7 Vorgehen und Projektorganisation

Wir arbeiten mit Scrum in Sprints von zwei Wochen. Ein Sprint beginnt und endet donnerstags. Am Ende jedes Sprints zeigen wir dem Auftraggeber im Sprint-Review den Stand. Die Reviews finden donnerstags statt, nach Absprache mit Prof. Dr. Rainer Neumann (Termine in Abschnitt 10). Aufgaben verwalten wir in einem gemeinsamen Jira-Board, Code in Gitlab und Dokumentation in Gitlab und Confluence. Leistungsort ist Karlsruhe (Hochschule und Arbeitsplätze der Teammitglieder).

7.1 Entscheidungen und Eskalation

- **Operative Rückfragen:** Ansprechperson der HKA und Projektteam, per Element oder E-Mail.
- **Fachliche Entscheidungen:** Prof. Dr. Rainer Neumann und Dr. Günther Schreiner.
- **Änderungen an Umfang oder Terminen:** gemeinsame schriftliche Freigabe nach Abschnitt 12.

- **Blockaden:** Eine Blockade ist ein Hindernis, das die Arbeit länger als zwei Werktage aufhält, zum Beispiel ein fehlender Zugang oder eine offene Entscheidung. Dann gilt:
- Unser Product Owner informiert die Auftraggeber (Prof. Dr. Rainer Neumann und Dr. Günther Schreiner) in Textform. Die Nachricht nennt das Hindernis, die Folge für die Termine und die gewünschte Entscheidung.
- Die Auftraggeber entscheiden innerhalb von drei Werktagen.
- Kommt keine Entscheidung, arbeiten wir an anderen Aufgaben weiter. Die betroffenen Termine verschieben sich um die Dauer der Verzögerung (Abschnitt 8). Wir halten Hindernis und Entscheidung im Jira-Board fest.

7.2 Team und Qualifikation

Das Team und seine Rollen stehen in Abschnitt 2. Unser Team arbeitet täglich mit KI-Assistenten. Wir kennen die Fehler, die KI-Code enthält, und sind darin geübt, Regeln dagegen zu bauen. Am 06.10. und 07.10.2026 haben wir die Architekturdiagramme nach dem C4-Modell (Level 1 und 2) erstellt. Mit den Startern beginnen wir im ersten Sprint, direkt nach der Angebotspräsentation und dem Kickoff am 22.10.2026.

Als kleines Team bieten wir kurze Wege und persönliche Betreuung. Den Preis erklärt Abschnitt 11.

8 Voraussetzungen und Mitwirkung

Unsere Planung beruht auf den folgenden Voraussetzungen. Die Tabelle nennt zu jeder Mitwirkung der HKA den spätesten Termin. Der Durchstich am 19.11.2026 hängt von den Leistungen ab, die bis zum 05.11.2026 vorliegen müssen. Verzögert sich eine Mitwirkung, verschieben sich die davon abhängigen Termine um denselben Zeitraum.

8.1 Mitwirkung der HKA

Mitwirkung der HKA	Nötig für	Spätestens bis
Ansprechperson je Thema (Login, Server, Mail) und für fachliche Entscheidungen, ohne Wartezeit	Entscheidungen und Abnahme	22.10.2026 (Kickoff)
Gitlab-Organisation oder Repository-Bereich der Hochschule, mit Rechten für alle Teammitglieder	Code, Pipeline und Freigabe von Pull Requests	29.10.2026
Server im HKA-Netz (Proxmox-VM mit installiertem Docker und SSH-Zugang)	Test, Deployment und Betrieb des Portals, Durchstich	05.11.2026
Shibboleth-Testumgebung (OIDC) mit Test-Accounts für mindestens drei Rollen	Portal-Login und Durchstich	05.11.2026
Offene Technikfragen klären (Zugriff auf Fachsysteme, Datenhaltung), gemeinsam mit dem Projektteam; Ergebnis schriftlich	Planung der Phasen 2 und 3	05.11.2026

Mitwirkung der HKA	Nötig für	Spätestens bis
Zugänge zu Test-Schnittstellen für Noten, E-Akte und Exchange	Schnittstellen und Referenz-Apps	26.11.2026
Teilnahme an den Sprint-Reviews (donnerstags, Termine in Abschnitt 10)	Abnahme je Phase	Termine reserviert bis 29.10.2026
Pilotpersonen (mindestens zwei Fachexpertinnen oder Fachexperten), Pilot in KW 50 bis KW 3	Test unter Praxisbedingungen	Benennung bis 03.12.2026
Eine Person der HKA-IT für den Übergabetest (Plattform anhand der Betriebsanleitung starten und neu starten)	Endabnahme	28.01.2027

9 Qualität, Erfolgs- und Abnahmekriterien

9.1 Qualitätssicherung und Definition of Done

Wir sichern die Qualität unserer Arbeit mit diesen Maßnahmen:

- **Eigene Pipeline:** Wir nutzen die Pipeline der Plattform auch für unsere eigene Arbeit.
- **Vier-Augen-Prinzip:** Mindestens eine zweite Person gibt jede Änderung frei.
- **Automatisierte Tests:** Wir testen mit pytest (Python) und Jest (JavaScript). Die Tests laufen bei jedem Pull Request in der Pipeline. Ein Pull Request mit roten Tests wird nicht gemergt.
- **Manuelle Prüfung:** Nachweise, die sich nicht automatisieren lassen (zum Beispiel Login mit Hochschul-Account oder das Aussehen), prüfen wir von Hand und halten das Ergebnis in einem Testprotokoll fest.
- **Kriterien vorab:** Abnahmenachweise (Abschnitt 6) und Abnahmekriterien je Phase (Abschnitt 9.3) stehen vor Beginn der Phase fest.
- **Sprint-Review:** Alle zwei Wochen zeigen wir den Stand und holen Rückmeldungen der Auftraggeber ein.

Eine Aufgabe gilt als abgeschlossen (Definition of Done), wenn diese Punkte erfüllt sind:

- Die Funktion ist automatisiert getestet, die Tests laufen in der Pipeline grün, und sie erfüllt den Abnahmenachweis aus Abschnitt 6.
- Ruff und ESLint finden keine Fehler, und mindestens eine zweite Person hat den Code freigegeben.
- Die Dokumentation (README, Architektur nach C4, Betriebsanleitung) ist auf dem aktuellen Stand.
- Es gibt keine kritischen Fehler, die Kernfunktion oder Sicherheit betreffen (siehe Mängelklassen in Abschnitt 9.4).

Die Sprint-Reviews alle zwei Wochen dienen als Kontrolle durch die HKA. Testergebnisse, Prüfprotokolle und Reviews halten wir im Jira-Board und in Gitlab fest.

9.2 Kennzahlen

Wir messen den Projekterfolg an diesen Zielwerten:

Kennzahl	Beschreibung und Messung	Zielwert
Erfolgsrate der Pipeline	Anteil der Merges in `main`, bei denen Secret-Scan, Manifest-Check, Lint, Tests und Docker-Build bestanden waren und eine zweite Person freigegeben hat (Auswertung der Gitlab-Historie am Ende jeder Phase)	100 %
Regeltreue der KI	Anteil der sieben Regeln, die die KI in der Testaufgabe einhält (Prüfskript)	7 von 7
Nutzung der Starter	Fachexpertinnen und Fachexperten bauen mit den Startern Apps und bringen sie ins Portal	Mindestens eine App aus dem Pilot im Portal und Live-Demos in der Abschlusspräsentation

9.3 Abnahme

Wir zeigen jede Phase im Sprint-Review und nehmen sie anhand der Abnahmenachweise aus Abschnitt 6 und der weiteren, vorab definierten Abnahmekriterien aus der folgenden Tabelle ab. Die Kriterien stehen mit diesem Angebot fest. Sie ändern sich nur nach Abschnitt 12.

Bei den Phasen 1 bis 3 erklärt der Auftraggeber die Abnahme innerhalb von 10 Werktagen nach dem Sprint-Review oder nennt die Mängel schriftlich. Äußert er sich innerhalb dieser Frist nicht, gilt die Phase als abgenommen (§ 640 Abs. 2 BGB). Die Endabnahme von Phase 4 findet im Anschluss an die Abschlusspräsentation am 28.01.2027 statt, spätestens am 29.01.2027.

Phase und Review	Abnahmekriterien (zusätzlich zu den Abnahmenachweisen aus Abschnitt 6)
1 Grundlagen Review 05.11.2026	• Die Architektur (C4 Level 1 bis 2) liegt im Repository. • Die Gitlab-Repositories sind angelegt. Eine leere App besteht in der Pipeline Lint und Tests. • Die sieben Regeln und die Testaufgabe sind schriftlich festgelegt. • Die Ergebnisse zu Hosting und Datenhaltung sind dokumentiert. • Verzögert sich eine Mitwirkung der HKA nach Abschnitt 8, gilt das davon abhängige Kriterium als erfüllt, sobald wir den Stand dokumentiert haben. Die Termine verschieben sich nach Abschnitt 8.
2 Durchstich Review 19.11.2026	• Beide Starter haben die Projektstruktur und die fünf Befehle. • Die Durchstich-App (Python-Starter) zeigt nach dem Login mit einem Shibboleth-Testaccount Name und Rollen an. • Ein Pull Request mit der Durchstich-App durchläuft alle Prüfungen und wird von einer zweiten Person freigegeben. Nach dem Merge läuft die App auf einer Test-VM und erscheint ohne manuelle Änderung im Portal-Menü. • Ein Pull Request mit einem absichtlich eingebauten Test-Secret wird von der Pipeline blockiert.

Phase und Review	Abnahmekriterien (zusätzlich zu den Abnahmenachweisen aus Abschnitt 6)
3 Ausbau Review 14.01.2027	• Alle Ergebnisse aus Abschnitt 6 (Golden Master, Portal, Pipeline, Sicherheit, Referenz-Apps) liegen vor und sind im Review vorgeführt. • Je Starter liegt ein Prüfprotokoll vor, und das Prüfskript meldet 7 von 7 Regeln bestanden. • Eine App lässt sich laut Betriebsanleitung starten, stoppen, neu starten, auf die vorherige Version zurücksetzen und beenden. • Die Pilotpersonen sind eingewiesen.
4 Abschluss Präsentation 28.01.2027	• Alle Ergebnisse aus Abschnitt 6 (Pilot, Dokumentation) liegen vor, einschließlich der Prüfprotokolle und des Prüfskripts. • Die Kennzahlen aus Abschnitt 9.2 sind ermittelt. • Übergabetest: Eine Person der HKA-IT startet die Plattform anhand der Betriebsanleitung, stoppt sie und startet sie neu. Wir greifen nur ein, wenn die Anleitung fehlerhaft ist.

9.4 Mängel

Wir ordnen Mängel in zwei Klassen ein:

Klasse	Was dazu gehört	Folge
Wesentlich	Ein Abnahmenachweis oder ein Abnahmekriterium ist nicht erfüllt. Oder: Sicherheit, Anmeldung, Deployment oder Start der Plattform sind betroffen. Oder: Eine Funktion fällt aus, und es gibt keinen zumutbaren Umweg.	Die Abnahme wird verweigert. Wir beheben den Mangel und zeigen ihn im nächsten Sprint-Review erneut.
Gering	Alle Abnahmenachweise und Abnahmekriterien sind erfüllt, und die Funktion ist nutzbar. Beispiele: Rechtschreibfehler in Texten oder Dokumentation, kleine Abweichungen im Aussehen (Abstand, Farbton), eine unklare Fehlermeldung.	Die Abnahme wird nicht verweigert (§ 640 Abs. 1 Satz 2 BGB). Der Auftraggeber hält den Mangel im Abnahmeprotokoll fest. Wir beheben ihn in der folgenden Phase, bei Phase 4 innerhalb von 10 Werktagen nach der Endabnahme.

10 Termin- und Meilensteinplan

Das Projekt läuft vom 22.10.2026 bis zur Endabnahme am 29.01.2027. Der erste Sprint beginnt mit der Angebotspräsentation und dem Kickoff; Voraussetzung ist die Annahme dieses Angebots bis zum 23.10.2026. Vom 21.12.2026 bis zum 01.01.2027 arbeiten wir nicht am Projekt. Ein Meilenstein gilt als erreicht, wenn die Liefergegenstände samt Abnahmenachweis und die Abnahmekriterien aus Abschnitt 9.3 vorliegen. Die Termine gelten, wenn die Mitwirkung aus Abschnitt 8 rechtzeitig erfolgt.

Sprint	Zeitraum	Sprint-Review	Phase
1	22.10. bis 05.11.2026	05.11.2026	1
2	05.11. bis 19.11.2026	19.11.2026	2
3	19.11. bis 03.12.2026	03.12.2026	3
4	03.12. bis 17.12.2026	17.12.2026	3
5	17.12.2026 bis 14.01.2027 (Pause 21.12. bis 01.01.)	14.01.2027	3
6	14.01. bis 28.01.2027	28.01.2027 (Abschlusspräsentation)	4

Phase	Zeitraum	Meilenstein
1 Grundlagen	22.10. bis 05.11.2026 (KW 43 bis 45, 2 Wochen)	Kickoff ist erfolgt. Architektur (C4 Level 1 bis 2), Repositories und Gitlab stehen. Die Mitwirkungen mit Termin bis 05.11.2026 (Abschnitt 8.1) liegen vor.
2 Durchstich	05.11. bis 19.11.2026 (KW 45 bis 47, 2 Wochen)	**Durchstich am 19.11.2026:** Der Golden Master steht (Projektstruktur und Befehle beider Starter). Die Durchstich-App aus dem Python-Starter geht per Pull Request in einer Testumgebung live und erscheint im Portal. Ihre Funktionen stehen in Abschnitt 9.3.
3 Ausbau	19.11.2026 bis 14.01.2027, ohne Pause 21.12. bis 01.01. (KW 47 bis 51 und KW 1 bis 2, 6 Arbeitswochen)	Skills, Gestaltungsregeln, Schnittstellen, Secret-Sicherungen, Portal und Security-Gateway sind fertig. Die KI-Tests laufen ab KW 47. Die Referenz-Apps werden ab KW 49 eingebunden. Der Pilot beginnt am 11.01.2027 (KW 2).
4 Abschluss	14.01. bis 29.01.2027 (KW 2 bis 4, 2 Wochen)	Der Pilot (KW 50 bis 3) ist ausgewertet. Dokumentation, Übergabe und Abschlusspräsentation am 28.01.2027 sind fertig. **Endabnahme am 29.01.2027.**

11 Vergütung

Der Festpreis für den Leistungsumfang beträgt **36.000 € zuzüglich gesetzlicher Umsatzsteuer**. Wir bieten einen Phasenfestpreis an: Jede Phase hat einen Festpreis. Der Festpreis bezahlt das Ergebnis (Werkvertrag). Er ändert sich nicht, wenn wir mehr oder weniger Stunden brauchen als kalkuliert.

11.1 Stundensatz und Kooperationsrabatt

Wir kalkulieren mit 900 Stunden (fünf Personen mit je 15 Stunden pro Woche über 12 Arbeitswochen) und einem **Normalsatz von 60 € pro Stunde**. Wir sind Studierende mit Erfahrung in KI-gestützter Entwicklung, aber ohne langjährige Projekterfahrung. Deshalb liegt unser Normalsatz bei den Sätzen für

Junior-Entwicklerinnen und Junior-Entwickler (45 bis 65 €) und deutlich unter dem Median für Freiberufler in der DACH-Region (90 €; Quellen in Anlage 3).

Auf den Normalpreis von 54.000 € geben wir der HKA einen **Kooperationsrabatt von einem Drittel (18.000 €)**. Daraus ergibt sich der Festpreis von 36.000 €, rechnerisch 40 € pro Stunde. Wir geben den Rabatt aus drei Gründen:

- Das Projekt ist zugleich unser Anwendungsprojekt im Studium. Wir sammeln Praxiserfahrung und können das Ergebnis als Referenz nennen.
- Die HKA stellt Server, Gitlab, Test-Login und Ansprechpersonen bereit.
- Wir haben keine Vertriebs- und Verwaltungskosten.

Phase	Wochen	Aufwand (h)	Normalpreis (60 €/h)	Rabatt (⅓)	Festpreis (€)
1 Grundlagen	2	150	9.000	−3.000	6.000
2 Durchstich	2	150	9.000	−3.000	6.000
3 Ausbau	6	450	27.000	−9.000	18.000
4 Abschluss	2	150	9.000	−3.000	6.000
Summe	**12**	**900**	**54.000**	**−18.000**	**36.000**

- **Rechnungsstellung:** je Phase nach der Abnahme, zahlbar innerhalb von 30 Tagen netto.
- **Zusatzaufwand:** Änderungen außerhalb des Leistungsumfangs rechnen wir nach schriftlicher Abstimmung zum Normalsatz von 60 € pro Stunde ab (siehe Abschnitt 12). Der Kooperationsrabatt gilt nur für den Festpreis dieses Angebots.
- **Nebenkosten:** keine. Wir nutzen Open-Source-Werkzeuge und die Infrastruktur der Hochschule. Reisekosten entstehen nicht.
- **Ausstieg:** Es gilt das gesetzliche Kündigungsrecht des Auftraggebers (§ 648 BGB). Nach jeder abgenommenen Phase kann die HKA entscheiden, ob sie weitermacht.

12 Änderungsverfahren

Wir nehmen Änderungswünsche und Wünsche nach neuen Features jederzeit auf. Den Inhalt eines laufenden Sprints ändern wir nicht. Änderungen wirken ab dem nächsten Sprint. So bleiben die zugesagten Ergebnisse des Sprints planbar. Das Verfahren:

1. **Wunsch melden:** Der Auftraggeber meldet den Wunsch dem Product Owner (per Element, E-Mail oder im Sprint-Review).
2. **Schätzen:** Wir schätzen den Aufwand in Stunden und nennen die Folgen für Termine. Die Schätzung erhält der Auftraggeber schriftlich innerhalb von fünf Werktagen. Das gilt auch für Features, die noch nicht im Leistungsumfang stehen.

3. **Entscheiden:** Der Auftraggeber wählt: **Tausch** (das Feature ersetzt noch nicht begonnene Leistung mit gleichem geschätzten Aufwand, der Festpreis bleibt gleich), **Zusatz** (das Feature kommt zusätzlich und wird zum Normalsatz vergütet, siehe Abschnitt 11) oder **Verzicht**.
4. **Festhalten:** Wir halten Tausch oder Zusatz mit Aufwand, Preis, Terminen und angepassten Abnahmenachweisen in einem Nachtrag fest. Der Auftraggeber gibt ihn in Textform frei.
5. **Umsetzen:** Wir setzen die Änderung ab dem nächsten Sprint um. Bis zur Freigabe arbeiten wir nach diesem Angebot weiter.

Ein Tausch ist nur für Leistung möglich, die noch nicht begonnen ist. Bereits abgenommene Phasen ändern wir nicht.

13 Übergabe und Weiterbetrieb

Die Übergabe findet in KW 3 bis 4 statt, vor der Endabnahme am 29.01.2027. Wir übergeben der HKA:

- den Quellcode in der Gitlab-Organisation der HKA
- die technische Dokumentation mit Architektur (C4 Level 1 bis 2)
- die Betriebsanleitung und die Deployment-Konfiguration
- die Anleitung für Fachexpertinnen zum Anlegen neuer Apps
- die Prüfprotokolle und das Prüfskript der KI-Tests

Danach kann die HKA-IT die Plattform anhand der Betriebsanleitung ohne das Projektteam starten, betreiben und erweitern. Das prüfen wir im Übergabetest (Abschnitt 9.3). Produktivbetrieb, Wartung und Support nach der Abnahme sind nicht enthalten. Wir können sie getrennt vereinbaren.

14 Angebots- und Vertragsbedingungen

14.1 Nutzungsrechte

Umfang: Der Auftraggeber erhält an allen von uns gelieferten Ergebnissen (Quellcode, Skills, Gestaltungsregeln, Konfiguration, Dokumentation und Bilder) ein einfaches, also nicht ausschließliches Nutzungsrecht im Sinne von § 31 Abs. 2 UrhG. Es gilt ohne zeitliche und räumliche Grenze. Er darf die Ergebnisse nutzen, vervielfältigen, ändern und weitergeben, auch an künftige Entwicklerinnen und Entwickler der Plattform und für alle Zwecke der HKA. Das schließt Apps ein, die Fachexpertinnen und Fachexperten mit den Startern bauen.

Zeitpunkt: Das Nutzungsrecht an den Ergebnissen einer Phase geht mit der Zahlung dieser Phase auf den Auftraggeber über. Bis dahin darf er die Ergebnisse zum Test und zur Abnahme nutzen.

Nutzung durch uns: Wir dürfen die Ergebnisse selbst weiter nutzen und weiterentwickeln, zum Beispiel für Studienarbeiten, unser Portfolio und spätere Projekte, und sie als Referenz nennen. Vertrauliche Informationen der HKA (Abschnitt 14.3) und Zugangsdaten sind davon ausgenommen. Marken, Logos und das Hochschul-CSS der HKA bleiben Eigentum der HKA. Wir nutzen sie nur im Projekt.

Drittkomponenten und KI-Code: Open-Source-Komponenten Dritter unterliegen ihren eigenen Lizenzen. Wir nennen sie in der Dokumentation. Teile der Ergebnisse entstehen mit KI-Assistenten. Besteht an solchen Teilen kein Urheberrecht, schränkt das die Nutzung durch den Auftraggeber nicht ein.

14.2 Gewährleistung und Haftung

Es gilt die gesetzliche Haftung für Mängel bei Werkverträgen. Eine weitere Garantie übernehmen wir nicht. Bei Vorsatz und grober Fahrlässigkeit haften wir ohne Grenze. Bei leichter Fahrlässigkeit haften wir nur bei Verletzung wesentlicher Vertragspflichten und bis zur Höhe des Festpreises. Die Haftung für Schäden an Leben, Körper und Gesundheit bleibt unberührt.

14.3 Geheimhaltung und Datenschutz

Wir behandeln alle internen Informationen der Hochschule vertraulich, auch nach Projektende. Personenbezogene Daten verarbeiten wir nur nach Vorgabe des Auftraggebers und nach DSGVO. Für Tests nutzen wir nur Testdaten oder eigene Daten der Teammitglieder.

14.4 Gültigkeit und Schlussbestimmungen

Dieses Angebot gilt bis zum 23.10.2026. Änderungen des Vertrags brauchen die Textform. Ist eine Bestimmung unwirksam, bleibt der übrige Vertrag wirksam. Es gilt deutsches Recht. Gerichtsstand ist Karlsruhe.

15 Anlagen

Anlage 1: Konzeptpräsentation „Micro-App-Plattform“ (21 Folien)

Anlage 2: Architekturbilder C4 Level 1 bis 2 (Abschnitt 5.5; Stand bei Angebot)

Anlage 3: Quellen zum Stundensatz: freelancermap, „Stundensatz Softwareentwickler“ (https://www.freelancermap.de/blog/?p=72121) und JobRise, „Freelance Developer Deutschland 2026“ (https://jobrise.io/de/blog/freelance-developer-deutschland-2026/)

Annahmeerklärung

Hiermit nimmt der Auftraggeber das Angebot **AWP-WS26-01** zum Festpreis von **36.000 € zuzüglich Umsatzsteuer** an.

Auftraggeber	**Auftragnehmer**
Ort, Datum	Ort, Datum
Name	Name
Unterschrift	Unterschrift